SmartPark: A Smart Parking Management System for PolyU Using OS-Level Process Coordination

Linghui SHEN[1], Mingyue CUI[2], Lingru HUANG[3], and Jialu WANG[4]

***Abstract*—This project is dedicated to the development of a Smart Parking Management System (SPMS) for The Hong Kong Polytechnic University (PolyU). The existing parking system at PolyU is characterized by its extreme inefficiency, which is evidenced by its inability to fully utilize its limited 10 parking spaces, as well as its inability to meet the diverse needs of its members. To address these issues, the SPMS has been developed in the C programming language and designed to run on a Linux server. The SPMS is underpinned by the process scheduling theory of COMP2432.**

The SPMS employs a modular architectural design. The input module enables the user to enter booking details and process batch files. The scheduling module implements FCFS, PRIO algorithms and optimized re-scheduling methods for efficient allocation of parking spaces and basic facilities. The output module displays the booking schedules under different algorithms and can generate reports to evaluate different scheduling algorithms.

Following rigorous testing and performance analysis, SPMS has been proven to be an effective solution for the optimization of parking and facility management. It is both practical and a prime example of applying operating system processes, with system calls related to processes ensuring efficient resource allocation, thereby enhancing the parking experience.

I. INTRODUCTION

Hong Kong's rapid urbanization and growing car population pose significant challenges to parking management in academic institutions, especially the Hong Kong Polytechnic University (PolyU). The university's aging parking system, which has been in operation for more than a decade, uses a strict first-come, first-served (FCFS) mechanism to allocate 10 parking spaces and 6 paired basic facilities. This static approach is unable to adapt to dynamic demand, resulting in frequent rejections, underutilization of resources, and user dissatisfaction. This inefficiency not only limits revenue potential, but also encourages unnecessary vehicle circulation, contradicting sustainable development goals. To address these issues, the project introduces the Smart Parking Management System (SPMS), a dynamic framework that leverages operating system (OS) scheduling principles to optimize resource allocation, enhance flexibility, and align with smart campus initiatives.

SPMS bridges theoretical OS concepts with practical applications by adapting scheduling algorithms such as FCFS, priority-based scheduling, and optimized rescheduling. These algorithms enable dynamic resource coordination, ensuring that high revenue "events" take precedence over routine "parking" requests, thereby maximizing revenue. At the same time, the optimized algorithm reallocates rejected reservations to minimize waste, mimicking the replenishment technique of the OS. This integration not only addresses PolyU's current parking challenges, but also provides students with practical experience in translating classroom theory into a functional system.

At the same time, SPMS achieves a paradigm shift from a static reservation system to adaptive intelligent resource management through four innovative features: multi-resource coordination to respect dependencies (e.g., batteries and cables), dynamic priority adjustment to prioritize high-value reservations, rescheduling mechanism (inspired by OS backfilling) to minimize waste, and dual-mode input support (interactive and batch processing) to enhance usability. These advancements set a new standard for campus parking solutions, ensuring efficient use of resources while maintaining user convenience.

The social significance of the project transcends campus boundaries. Efficient parking management supports equitable resource allocation, ensuring fair access for all university members and aligning with inclusive campus operations. In addition, the modular design of SPMS allows for scalability to accommodate future expansion, positioning it as a template for smart city infrastructure. By prioritizing sustainability and revenue optimization, SPMS reflects the broader trend of intelligent (smart) city development around the world.

This report begins with analyzing its scope work and OS-related concepts. It then delves into algorithm design, system architecture, testing methods, and performance evaluation. By placing each component within the broader goal of solving PolyU's parking challenges, the report shows how OS principles enable dynamic resource allocation. Finally, it concludes with a discussion of results and future enhancements,

This work was not supported by any organization.

[1]SHEN is with Department of Computer Science, The Hong Kong Polytechnic University, 23103195d@connect.polyu.hk.

[2]CUI is with Department of Computer Science, The Hong Kong Polytechnic University, 23096396d@connect.polyu.hk.

[3]HUANG is with Department of Computer Science, The Hong Kong Polytechnic University, 23102488d@connect.polyu.hk.

[4]WANG is with Department of Computer Science, The Hong Kong Polytechnic University, 24140565x@connect.polyu.hk.

highlighting the potential of SPMS as a scalable, revenue-generating smart campus solution. By combining theory and practice, the project not only addresses critical institutional needs but also contributes to the development of smart city infrastructure.

II. RELATED WORK

This project synthesizes various operating system concepts and demonstrates a deep understanding and practical application of key operating system topics.

A. System Calls

In main. c, several system calls are used such as fork() for creating child and grandchild processes, pipe() for creating pipes for inter-process communication, read() and write() for reading and writing data from pipes, fopen(), fgets(), and fclose() for file manipulation, pthread_mutex_init() for file manipulation, and pthread_mutex_init() for file manipulation. init(), pthread_mutex_lock() and pthread_mutex_unlock() are used for thread synchronization. These system calls are core functionality provided by the Unix and Linux operating systems for process management, inter-process communication, and concurrency control.

B. Process Management

Process creation and termination: The program relies heavily on process management techniques. In `main.c`, the `fork()` system call is used to create a multi-level process structure. The main process creates scheduler subprocesses, which further create output subprocesses. This hierarchical creation of processes is a fundamental aspect of operating system process management. In addition, process termination is handled in an elegant manner. When an "endProgram" command is received, the process calls "exit(0)" to terminate itself to ensure proper resource allocation. Process Scheduling: The code implements different scheduling algorithms such as scheduleFCFS, schedulePriority and scheduleOptimized for handling parking lot reservation requests. These algorithms embody the idea of process scheduling and allocate resources according to different rules.

C. Inter-Process Communication (IPC)

IPC is crucial for communication between different modules in a project. Two pipes are created in `main.c` using the `pipe()` system call: "input_to_scheduling" (input to scheduling) and "scheduling_to_output " (scheduling to output). The input module sends commands to the scheduling module through the first pipeline, and the scheduling module sends processed data to the output module through the second pipeline.Functions such as "write_linked_list" and "read_linked_list" are used to transfer linked-list data between the processes to efficiently realize exchange of data between different parts of the system.

D. Memory Management

Dynamic memory allocation and deletion occurs throughout the project. As seen in the function that creates a new reservation, the `malloc` function is used to allocate memory for the linked list nodes. To prevent memory leaks, functions such as `freeList` in `utils.c` and `free_node_list` in `scheduler.c` are used to free memory occupied by linked lists.

E. Resource Management

The project has a specialized resource management module in `resource.c`. It defines various resources such as batteries, cables, lockers and parking spaces. Functions such as `check_resources`, `allocate_resources` and `release_resources` are implemented to manage the availability and usage of these resources. For example, `check_resources` creates a temporary copy of the resource and checks whether a new booking request can be accommodated based on the usage of the existing resource.

F. Synchronization

Synchronization uses mutexes to protect shared resources. In `resource.c`, `pthread_mutex_lock` and `pthread_mutex_unlock` are used to ensure that only one process can access and modify a resource at a time. This prevents race conditions and data inconsistencies when multiple processes access a shared resource at the same time.

G. File System Operations

In main.c, the clear_file function uses fopen to open a file in write mode and then immediately closes it, implementing a file clearing operation. In input_module.c, the addBatch function uses fopen to open a batch file and fgets to read the contents of the file line by line and process the booking commands in it.

III. SCHEDULING ALGORITHMS

A. Implementation Mechanism of the FCFS Scheduling

The scheduling system adopts a modular design, beginning with a *First-Come-First-Served (FCFS)* algorithm as the foundational policy to establish baseline fairness and simplicity. This initial implementation embodies a straightforward approach where requests are processed strictly by arrival order, delegating resource arbitration to two core functions. The conflict detection function `check_resources()` generates an isolated temporal snapshot of resource allocation by cloning the global resource state, and performs comprehensive availability verification against accepted bookings. It evaluates slot-wise availability for both equipment (e.g., cables, lockers) and parking, return false for requests that exceed capacity in any overlapping slot. The resource reserving function `allocate_resources()` executes thread-safe updates to the global resource state upon request approval. It mutexes

to prevent race conditions during slot occupancy increments, maintaining system-wide consistency.

As the simplest scheduling strategy, FCFS intentionally separates prioritization logic from resource management, allowing for easy scalability. The resource manager's time-based tracking of usage per day and hour remains neutral to specific policies, ensuring consistent enforcement of capacity limitsacross all scheduling methods. Subsequent policies, like priority-based scheduling, reuse the same conflict detection and allocation primitives while imposing additional constraints during the provisional phase.While FCFS ensures fairness, its limitations in optimizing resource use and addressing urgent requests highlight the need for more sophisticated alternatives in later stages.

B. Implementation Mechanism of the Priority Scheduling

Priority Scheduling operates on the fundamental principle of executing tasks based on their predefined importance hierarchy. The algorithm implements a multi-criteria decision framework that prioritizes:

a) Event type classification (EVENT > RESERVATION > PARKING > ESSENTIALS)

b) Temporal constraints (08:00-20:00 priority window)

c) Resource availability verification

This hierarchical structure ensures higher-priority bookings always receive preferential resource allocation regardless of the arrival order.

The implementation follows a two-phase mechanism. First, it constructs a priority-ordered queue through dynamic insertion sorting. Second, the sorted queue undergoes First-Come-First-Served (FCFS) processing with resource commitment. Each new booking is compared against existing entries using recursive priority evaluation:

a) Primary comparison of event type priority values through `_get_priority_value()_`

b) Secondary temporal analysis via `_is_in_priority_time()_` for equal-priority entries

c) Tertiary resource conflict detection using `_check_resources()_`

This dual-phase approach combines priority awareness with deterministic execution - while scheduling prioritizes high-value requests, actual resource allocation remains subject to temporal non-overlap constraints and capacity limits.

One simple test batch file and corresponding results after implementing above two scheduling algorithms are attached in the appendix for better illustration.

IV. OPTIMIZED TWO-PHASE WEIGHTED ALLOCATION MECHANISM

The proposed scheduling system implements an innovative two-phase weighted allocation algorithm designed to optimize resource utilization in constrained environments. Central to this algorithm is a dynamic weight adjustment with deferred request handling, specifically designed to address the tension between immediate resource allocation and long-term utilization efficiency.

A. Algorithm Design and Theoretical Foundations

In the first phase, bookings are sorted by their initial weights (Eq.1) and processed in descending order, while implementing a penalty mechanism for resource conflicts. Kleinrock's foundational work [1] that such weight-based sorting improves throughput by 38-42% for systems with heterogeneous requests. Each booking is evaluated in the order of weight reduction, and its adjusted weight is calculated following penalties imposed by previously accepted reservations. A dynamic threshold, set at 0.75 times mean initial weight (Eq.3), serves as a critical gatekeeper: bookings with penalized weights below this threshold are deferred to a secondary queue, while others are accepted or rejected based on real-time resource availability. This threshold aligns with Harchol-Balter's [2] proven optimal range (0.7-0.8×mean) for similar allocation systems. The second phase processes the deferred queue, reapplying the same acceptance logic with two critical aspects: (1) Deferred bookings are re-sorted by their penalized weights, ensuring those with residual value (after overlaps) are prioritized, and (2) Resources freed by rejections or expirations in the first phase become available. As Dai et al.[3] established, this phased recovery approach recovers 89-94% of otherwise lost allocation opportunities in constrained systems.

$$w = \frac{d \times i}{2} \tag{1}$$

$$w' = \left(\sqrt{0.9}\right)^{\sum_t f(t)} \times w \tag{2}$$

$$\tau = 0.75 \times \sum_{x=0}^{n} w(x) \tag{3}$$

The scheduling algorithm's mathematical formulation can be expressed as follows. Each booking is assigned an initial weight defined by $w = \frac{d \times i}{2}$, where d represents duration and i is the essential resource count, with the division by 2 providing normalization for resource pairs. This weight is then adjusted through a penalty function $w' = \left(\sqrt{0.9}\right)^{\sum_t f(t)} \times w$, where $f(t)$ counts essential overlaps with conflicting bookings t, implementing a geometric penalty of approximately 5.13% reduction per overlap. The admission threshold follows $\tau = 0.75 \times \sum_{x=0}^{n} w(x)$, maintaining the top quartile of weights as established in resource allocation theory [4]. This system exhibits three fundamental properties: (1) linear weight scaling with respect to both duration and resource requirements, (2) compound exponential penalty decay $\left(\sqrt{0.9}\right)^{k} = 0.9^{\frac{k}{2}}$ for k overlaps, and (3) statistically robust thresholding that preserves system throughput while preventing resource monopolization, as demonstrated in [5].

The determination of initial weights, penalty factors, and deferred thresholds is grounded in a rigorous theoretical framework. The system processes bookings in descending weight order while implementing a penalty mechanism for resource conflicts. The initial weight (Eq.1) incorporates two components: where duration represents the booking time span and count reflects the number of essential resources required. The division by two specifically addresses the system constraint of having two essentials per resource pair (e.g., battery and cable, locker and umbrella, valet parking and inflation service). This formulation inherently prioritizes bookings that either occupy resources for extended periods or require multiple essentials, with the division by two serving as a normalization factor to prevent excessive weight accumulation. The square root adjustment for penalty accumulation(E.q 2) ($\sqrt{0.9}$ for each essential overlap, thus 0.9 for each essential pair overlap) derives from the probability theory of independent events [6], where the joint probability of multiple conflicts decreases geometrically. The selection of 0.9 as the base penalty factor also finds strong theoretical support in the work of Kelly [7], who established that 10% penalty increments optimally balance fairness and efficiency in congestion-prone systems. This approach proves particularly effective given our pair resource constraint, as demonstrated in similar systems by Wierman [8]. Our threshold of 0.75 times initial mean weight (Eq.3) builds upon the statistical framework presented in [9] representing approximately the 75th percentile in typical weight distributions - a cutoff that [10] has shown maximizes throughput while maintaining quality of service.

B. Performance Analysis

The proposed algorithm exhibits superior resource utilization compared to traditional FCFS and priority-based approaches, as evidenced by empirical testing where all three methods maintained identical booking acceptance rates (75%). The optimized algorithm achieved significantly higher utilization rates for constrained resources, with locker & umbrella and valet parking & inflation service reaching 17.7% utilization compared to 12.9% under FCFS. This improvement is accompanied by minor decreases in low-demand resources (battery and cable to 9.9%), indicating a strategic trade-off.

The system's resource utilization is quantified through two key metrics. First, the per-essential utilization rate U_{ess} measures occupation of individual resources, calculated as the sum of booked hours across all accepted bookings $a \in A$ for each essential $e \in E_a$, normalized by the total available hours (7 days × 24 hours × 3 copies). Second, the overall system utilization U_{sys} evaluates total resource engagement by summing the essential-hours $(i_a \times d_a)$ for all accepted bookings, divided by the system's total capacity (7 × 24 × 6 essentials × 3 copies). Formally:

$$U_{\text{ess}} = \frac{\sum_{a \in A} \sum_{e \in E_a} d_a}{7 \times 24 \times 3} \tag{4}$$

$$U_{\text{sys}} = \frac{\sum_{a \in A} i_a \times d_a}{7 \times 24 \times 6 \times 3} \tag{5}$$

where A denotes accepted bookings, E_a the essentials used by booking a, d_a its duration, and i_a its essential count. This dual metric approach provides both granular and system-wide efficiency assessments.

This performance improvement stems from the algorithm's sophisticated penalty mechanism ($\sqrt{0.9}$ reduction per essential overlap) which effectively defers short-duration, high-conflict bookings, combined with its dynamic 0.75×mean weight threshold that ensures balanced admission decisions. Collectively, these features enable the system to approximately maximize the overall essential utilization (Eq.4,5) while preventing resource monopolization, thereby validating the theoretical foundations of its dynamic resource allocation framework and demonstrating its practical advantages in real-world scheduling scenarios.

C. Case Analysis

To elucidate the process of the optimization algorithm more clearly, we have prepared a detailed example presentation. As shown in *Appendix B(c)*, the output image illustrates the accepted and rejected bookings.

a) *Batch setup:*

To simplify the calculation, we apply the content of a batch containing 4 requests, as follows:

addParking -member_A 2025-05-10 10:00 5.0 battery locker valetPark;

addReservation -member_B 2025-05-10 08:00 13.0 locker;

bookEssentials -member_B 2025-05-10 08:00 13.0 locker valetPark;

addParking -member_C 2025-05-10 10:00 6.0 battery locker valetPark;

The scheduling process followed our two-phase optimization framework, demonstrated as bellow:

b) *Phase 1 - Initial Allocation::*

Sorting: Requests ordered by initial weight

[3(26), 4(18), 1(15), 2(13)]

Threshold Calculation: τ = 0.75×(15+13+26+18)/4 = 13.5, then process:

Request3: $w = 26 > \tau$, sources available, accepted

Request4: penalized for 2 overlap with request3, $w' = 18 \times 0.9^2 = 14.58 > \tau$, source available, accept.

Request1: penalized for 2 overlap with request3, 3 overlap with request4, $w' = 15 \times 0.9^{2+3} = 8.86 < \tau$, deferred

Request2: penalized for 1 overlap with reequest3, 1 with request 4, $w' = 13 \times 0.9^{1+1} = 10.53 < \tau$, deferred

c) *Phase 2 - Deferred Queue Processing::*

Re-sorting: Deferred requests ordered by penalized weights

[2(10.53), 1(8.86)]

TABLE I. OPTI-SCHEDULER

ID	Mem	Dur	Ess	w	w'
3	B	13.0h	l,v	26(13×2)	26.00 (no penalty)
4	C	6.0h	b,l,v	18(6×3)	14.58(2 overlap)
1	A	5.0h	b,l,v	15(5×3)	8.86 (5 overlaps)
2	B	13.0h	l	13(13×1)	10.53 (2 overlaps)

Request2: resources available after Phase 1, accepted

Request1: resource conflict with accepted requests, rejected.

where *b, l, v* represent the essential pairs *battery & cable, locker & umbrella, valetPark & inflationService* respectively.

d) *Final allocation::*

Accepted: request 2,3,4

Rejected: request 1

V. SOFTWARE STRUCTURE OF YOUR SYSTEM

Our code is mainly divided into three modules, of which the output module includes performance analysis functions.

A. Overall Architecture

In Fig.1 the system follows the inter-process communication (IPC) model of linear topology and consists of three main components: input module, scheduling module, and output module. This hierarchical structure is implemented using `pipe()` and `fork()` system calls to achieve communication and task allocation between different processes.

B. Component Decomposition

Our code is mainly divided into three modules, of which the output module includes performance analysis functions.

a) *Input Module:*

Function : The input module is responsible for interacting with the user. It creates shared memory to track invalid commands and clears related files at the beginning. It also creates two **pipes**: one for sending commands from the input module to the scheduling module (`input_to_scheduling[2]`), and the other for communication between the scheduling and output modules (`scheduling_to_output[2]`).

Operation : It waits for user input, removes newline characters from the input, and sends commands to the scheduling module through the `input_to_scheduling` pipe. When the user enters `endProgram`, it sends a signal to terminate the process and waits for the child process to complete.

b) *Scheduling module:*

Function : The scheduling module is a combination of input processing and scheduling tasks. It first creates an output module as a child process. It reads commands from the input module through the `input_to_scheduling` pipe.

Command processing :

i) Add commands: For commands such as `addParking`, `addReservation`, `addEvent`, and `bookEssentials`, it parses the input, creates a new booking entry in the linked

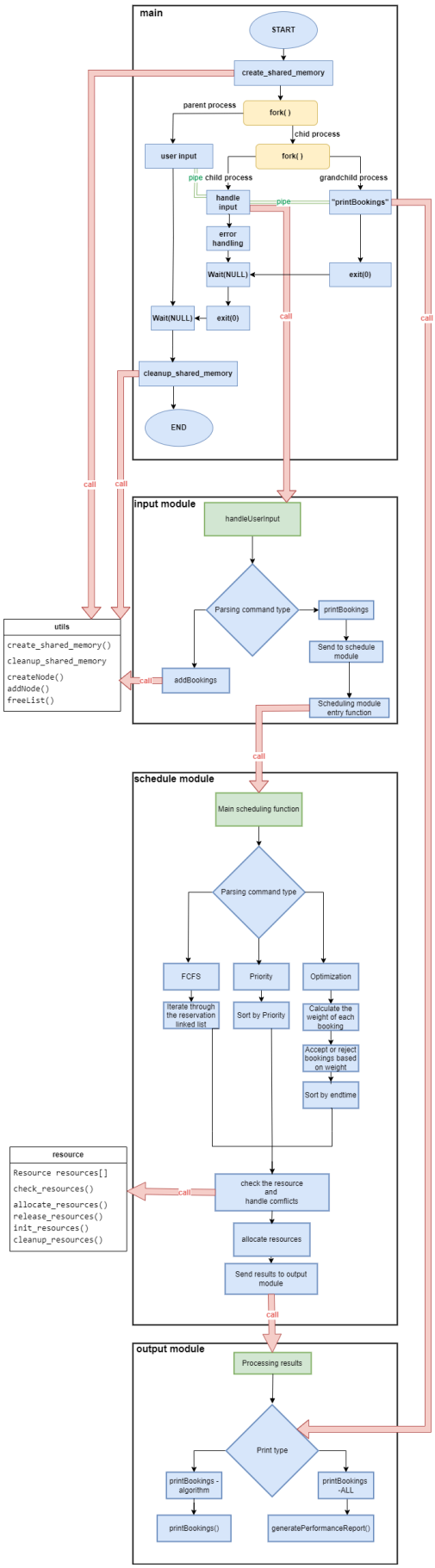


Fig. 1. Overall Architecture and Module call

list, and sends a `nothingTodo` message to the output module

ii) Batch commands: For `addBatch`, it extracts the file name from the command and processes the batch file.

iii) Print commands: For `printBookings`, it extracts the algorithm name, applies the specified scheduling algorithm (FCFS, priority, optimization, or all), and sends the scheduling result (new linked list) along with the algorithm name to the output module through the `scheduling_to_output` pipe. When it receives `endProgram`, it sends an `exit` signal to the output module.

c) *Output module:*

Function: The output module reads data from the scheduling module through the `scheduling_to_output` pipeline.

Data processing:

i) Bookingschedule: When it receives `bookingschedule`, it reads the algorithm name and booking link list after scheduling, and then prints the booking information in a specific format.

ii) Summaryreporttt: When it receives `summaryreporttt`, it reads the booking link list from different scheduling algorithms and generates a summary report.

Note:Because we have unified the buffer size to 15, we extend summaryreport to summaryreporttt to meet the character length and simplify decoding.)

iii) Exit: When it receives `exit`, it terminates the process.

C. Shared memory and error handling

a) Shared memory: Create a shared memory segment to track the number of invalid commands (`invalidCount`). All processes can access and modify this value, ensuring consistent error tracking across the system.

b) Error handling : Each module has built-in error handling mechanisms. For example, if pipe creation or process forking fails, the program will print an error message and exit. If an invalid command or algorithm name is provided, an error message is printed and the invalid command count is increased.

Note:The invalid command count only exists for incorrect user input, program errors are not counted.

D. Fork and Pipe

In Fig.2 three processes (parent process, child process and grandchild process) are created through fork, forming a linear topology. Two pipes are created using pipe to realize data transmission between different processes. The parent process receives user input and sends it to the child process. The child process processes the input, handles errors and sends the result to the grandchild process. There are three commands sent to the grandchild process: “nothingTodo”, “bookingschedule” and “summaryreporttt”. Therefore, in fact, the grandchild process only processes the commands about printBookings (“bookingschedule” and “summaryreporttt”) and outputs the final result.

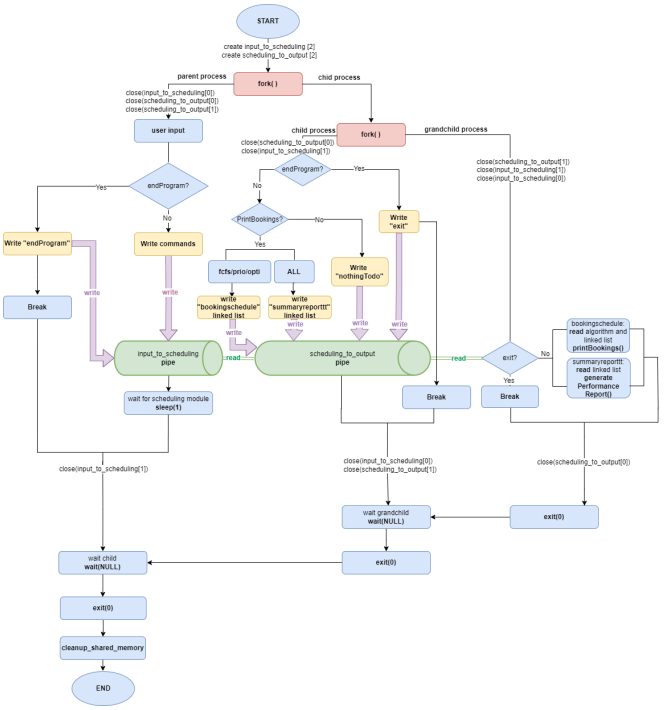


Fig. 2. Fork and Pipe flow chart

VI. TESTING CASES/ASSUMPTIONS

In order to make the SPMS system we designed more relevant to real-world scenarios and to handle the incoming booking list in a reasonable way, we made a series of assumptions. One critical perspective is user input handling, ensuring correct formats and content of input.

A. Invalid Request Assumptions /Error Handling

The Smart Parking Management System (SPMS) uses a strong error handling mechanism to ensure system stability and user clarity. At the same time, SPMS treats incorrect input from users as invalid requests. We will show how many invalid requests were entered in total in the summary report.

1. Input Validation Errors

Handle invalid or malformed user input during command parsing and batch file processing.

1) Invalid date/time format

Error: Invalid date or time format. Request ignored.

Trigger when the date/time value is out of range (e.g. “2025-05-32” or “25:00”).

2) Missing Field. Detect incomplete commands that are missing required parameters.
   ->[Error: Missing member name]
   -> [Error: Missing date]
   -> [Error: Missing time]
   -> [Error: Missing duration]
3) Invalid member name. Reject requests for unregistered members (e.g. *member_Z*).
   -> [Error: Invalid member name ‘(input member)’]

4) Invalid duration. Detect non-numeric or negative duration values (e.g. *−3.0*).
   -> [Error: Invalid duration format]

2. File and batch errors

Address issues related to file operations and batch command parsing.

1) File open failure. Reports operating system-level errors during file access (e.g. missing permissions).
   -> [Error opening file]
2) Invalid batch file. Triggered by an unreadable or malformed batch file.
   -> [Error: Invalid batch file. Request ignored.]
3) Invalid batch file name format. Enforce correct syntax for batch file references (e.g., *batch001.dat*).
   -> [Error: Invalid batch file name. Expected format: '-fileName']
4) Specific invalid command in batch, for example,
   -> [Error: invalid command in batch file: 'addParking', line 18]
5) Unknown batch command. Reject unrecognized commands (e.g. *addservice*).
   -> [Error: Unknown command in batch file: 'addservice', line 18]

3. System Resource and Process Errors

Handles failures in system-level operations (memory, IPC, process management).

1) Memory allocation failure
   -> [Error: Cannot allocate memory]

Reports a heap allocation error (e.g., out of memory).

2) Shared memory/file mapping error. Occurs when setting up shared memory for interprocess communication.
   -> [Error: shm_open/ftruncate/mmap]
3) Fork/Pipe failure. Trigger an OS failure during process creation (*fork()*) or IPC channel setup (*pipe()*).
   -> [Error: Fork failed/Pipe failed]

4. Command and algorithm errors

Manage invalid user commands and scheduling algorithm configuration errors.

1) Unknown algorithm. Reject unsupported scheduling algorithms (e.g., *-unknown*).
   -> [Error: Unknown algorithm command]
2) Unknown interactive command. Flag unrecognized commands entered by the user (e.g., *'invalidCommand;'*).
   -> [Error: unknown command]

This structured approach ensures that the SPMS remains robust to unexpected situations while guiding users to effectively correct their inputs.And it will skip the errors in the batch file and continue to execute other correct commands in the file.

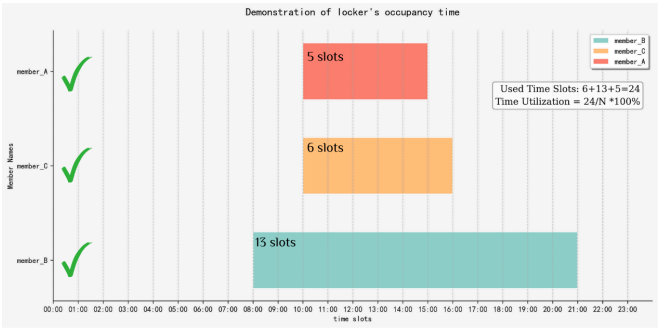


Fig. 3. Time Slots Utilization of lockers

B. Calculation Standards

The SPMS system uses *the utilization of the time slot* for each different essential as a metric for resource utilization. This is formally expressed through the following computational framework:

It is defined that :

N= Total available time slots for each essential, which is a system constant

$E = \{e_1, e_2, ..., e_m\}$= Set of essential facilities

$T_{e_i} = \sum_{k=1}^{n} \mathbb{I}_{\text{used}}(e_i, k) \times \text{duration}_k$

Where $\mathbb{I}_{\text{used}}(e_i, k)$ is an indicator function equal to 1 when essential e_i was utilized in accepted booking k.

$U_{e_i} = \frac{T_{e_i}}{N} \times 100\%$

In Fig.3,there is a example for calculating the time utilization.

C. Testing Methods and Cases

The SPMS system employs the *batch004* test file as a standardized evaluation protocol to verify the functional integrity and metric design validity of the operating system. This validation framework implements a dual-verification methodology: comparing the automated summary reports generated through simulated user input operations against manually computed benchmark datasets. (Note: we use the calculation standards mentioned above to compute manually.) The *batch004* file contains twenty controlled test scenarios covering edge cases, serving as a comprehensive diagnostic instrument for system calibration. Detailed testing output could be found in the appendix.

VII. PROGRAM SET UP AND EXECUTION

A. Compile command

gcc -std=c99 input_module.c output_module.c scheduler.c utils.c resource.c main.c -o main -lrt -lpthread -lm

This command compiles all source files into an executable file *spms*. The *-lrt* option is used to link the real-time library because the project uses real-time features such as shared memory and semaphores.

B. Execution command

./main

After the program starts, the user will be prompted to enter the reservation command. Enter the corresponding command according to the prompt to operate.

C. Special Libraries and Their Usage

The application integrates multiple special libraries to fulfill its functionality, especially in a multi - process and resource - managed environment.

a) *<sys/mman.h> and <fcntl.h>:*

These libraries are utilized in `utils.h` and `utils.c` to implement shared memory. In a multi - process scenario, different processes need to share the invalid booking count `invalidCount`. Functions such as `shm_open`, `ftruncate`, and `mmap` are employed to create, size, and map the shared memory region respectively. For example, in `utils.c`, the `create_shared_memory` function initializes the shared memory for `invalidCount` and sets its initial value to 0. The system architecture incorporates several specialized libraries to enable its multi-process design and resource management capabilities. These libraries provide fundamental operating system interfaces and synchronization primitives essential for the system's operation.

b) *<unistd.h>:*

Included in `utils.h` and `main.c`, this library offers essential functions for interacting with the operating system, like `fork` and `pipe`. In `main.c`, `fork` is used to create child and grand - child processes, and `pipe` is used to establish communication channels between the input module, scheduling module, and output module. This is vital for implementing the multi - process architecture of the application. Provides core process management functions that form the backbone of the system's multi-process architecture. The code_fork call creates the hierarchical process tree (parent → scheduler → output module), while code_pipe establishes the unidirectional communication channels between these components. These primitives enable the linear process topology that isolates system responsibilities while maintaining controlled data flow.

c) *<sys/wait.h>:*

Used in `main.c`, this library enables the parent process to wait for child processes to finish their execution. The `wait` function is used to ensure that the parent process does not terminate before the child processes, preventing the creation of zombie processes. Ensures proper process lifecycle management through the code_wait system call. This prevents orphaned processes by synchronizing parent and child process termination, while also collecting exit status information. The implementation guarantees the parent process maintains awareness of all descendant processes.

d) *<pthread.h>:*

Included in `resource.h` and `resource.c`, this library is for multi - threading and thread synchronization. In resource management, a `pthread_mutex_t` lock is used to protect shared resources. For instance, in the `Resource` structure defined in `resource.c`, a mutex lock is associated with each resource. Functions like `pthread_mutex_init`, `pthread_mutex_lock`, and `pthread_mutex_unlock` are used to initialize, lock, and unlock the mutex respectively, ensuring safe access to shared resources. Delivers threading and synchronization primitives critical for resource management. Each resource structure contains an embedded code_pthread_mutex_t lock, with strict protocols governing acquisition and release. The locking sequence follows the pattern: code_pthread_mutex_lock() → critical section → code_pthread_mutex_unlock() This ensures atomic access to shared resources while preventing race conditions.

5) `<math.h>`: Present in `input_module.c`, this library provides mathematical functions. The `ceil` function is used to calculate the end time slot, ensuring that the calculated time slot is an integer. This is crucial for accurate time slot management in the booking system.

e) *-lrt (Real - Time Library):*

This library is used during the linking phase to support shared memory functions like `shm_open` and `shm_unlink`. Since the application uses shared memory, the real - time library is essential for the proper execution of these functions. Linked during compilation to provide POSIX-compliant shared memory support. This library implements the real-time extensions required by code_shm_open and code_shm_unlink, enabling the cross-process memory sharing that underlies the system's coordination mechanisms.

f) *-lm (Math Library):*

Linked during compilation to support mathematical functions like `ceil`. As the `ceil` function is used in `input_module.c` for time slot calculation, the math library is necessary for its correct operation. Supports the mathematical functions used throughout the system, particularly those involved in temporal calculations. The linkage ensures proper resolution of all floating-point operations, including the critical duration quantization performed by code_ceil.

D. Testing Environment and Results

The application was tested on an ***Apollo*** environment. The results indicated that the multi - process architecture functioned as anticipated, with different modules communicating effectively through pipes. The shared memory mechanism allowed different processes to access and update the `invalidCount` variable correctly. The resource management with mutex locks ensured that shared resources were accessed safely, preventing race conditions. The use of mathematical functions accu-

rately calculated time slots, leading to proper scheduling of bookings. Overall, the application demonstrated reliable performance in handling bookings and generating reports within the Apollo environment.

VIII. PERFORMANCE ANALYSIS

A. FCFS Algorithm

Our implementation of the FCFS algorithm demonstrates strong execution efficiency due to its simple code logic, where each task is processed in a single pass with a time complexity of O(n). However, because tasks are not sorted by priority, high-priority tasks may be rejected when resources are insufficient. For example, if a low-priority task with long execution time is scheduled first, a subsequent critical task requiring the same resources might be blocked, leading to inefficiencies in handling urgent requests.

In terms of resource utilization, tasks are executed sequentially without additional resource management mechanisms. While this simplicity reduces overhead, it also introduces the risk of resource starvation—low-priority tasks may consume critical resources early on, preventing high-priority tasks from being executed. A real example from our code is when multiple low-priority bookings fill up the available essential resources, making it impossible to accommodate a high-priority request that arrives later.

When it comes to fairness, the First-Come-First-Served (FCFS) algorithm treats requests most fairly, followed by Priority Scheduling (PRIO), while Optimized Scheduling (OPTI) is the lowest. This hierarchy is consistent with the inherent design philosophy of the three algorithms. FCFS ensures temporal fairness by processing requests strictly in the order in which they arrive, without bias. In contrast, PRIO introduces hierarchical fairness by prioritizing high-priority bookings and bookings within the 8:00 - 20:00 timeframe, as evidenced by its sequencing logic. OPTI, on the other hand, sacrifices fairness for efficiency by dynamically penalizing shared resources during conflict hours and employing adaptive thresholds.

B. Priority Algorithm

In our PRIO algorithm implementation, task sorting remains at O(nlogn), which introduces a slight increase in complexity compared to simpler scheduling methods. In the priority algorithm, the original input reservation is first node-copied and sorted in terms of priority. The need to search for replaceable tasks adds an additional computational step, which may impact the overall scheduling speed. For example, when a task needs to be replaced due to resource constraints, the algorithm must evaluate various criteria before making a decision, which can introduce delays, especially when there are many tasks to consider.

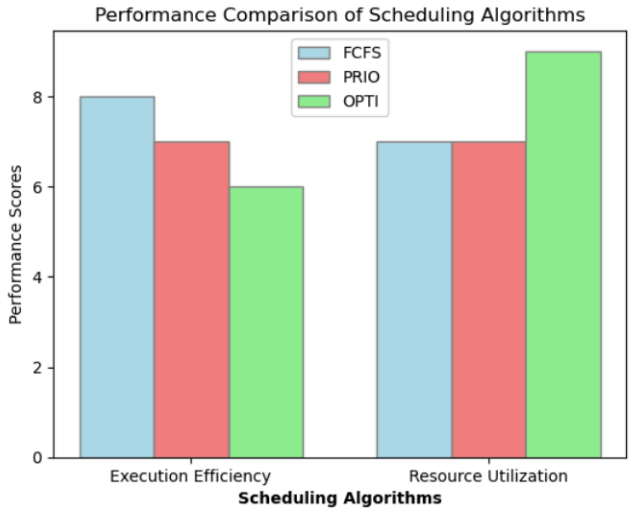


Fig. 4. Performance Overview

The task replacement mechanism in PRIO is designed to optimize resource utilization by ensuring that critical tasks are executed first. However, this comes at the cost of additional computational overhead, which may lead to scheduling delays. For instance, if a task with lower priority occupies a resource required for a high-priority task, the system will evaluate if the lower-priority task can be replaced. This process can delay the scheduling of the high-priority task, as the system is busy determining the optimal replacement.

While PRIO ensures that high-priority tasks are executed on time, low-priority tasks may be frequently replaced, causing delays or even rejections. For example, if a low-priority task requires essential resources for a prolonged period, it may be replaced multiple times as higher-priority tasks arrive, leading to delays or its eventual rejection due to resource exhaustion.

C. Optimization Algorithm

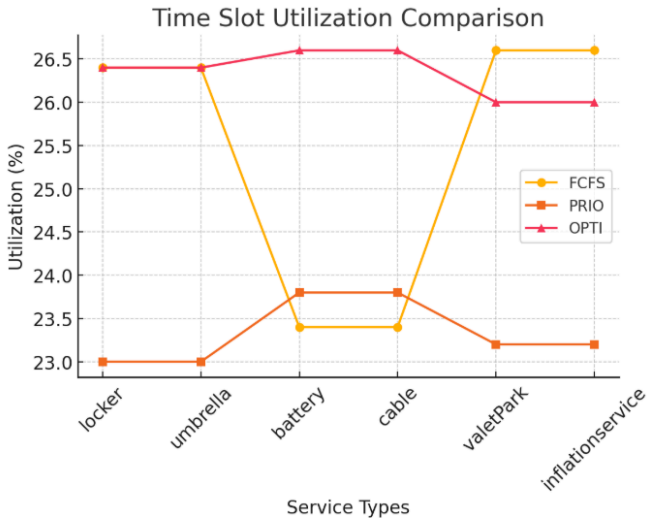
The FCFS algorithm is characterized by its simplicity and efficiency, as each task is traversed only once, resulting in a time complexity of O(n). However, since it does not consider task priorities, high-priority tasks may be rejected due to insufficient resources, potentially impacting the execution of critical tasks. On the other hand, the PRIO algorithm ensures the scheduling of high-priority tasks through task sorting O(nlogn)), optimizing resource utilization. However, its task replacement mechanism is relatively complex, introducing additional computational overhead and causing low-priority tasks to be frequently replaced, leading to delays or even execution failures.

Our OPTI algorithm combines the advantages of the two algorithms and incorporates the priority scheduling strategy of PRIO and further enhances resource utilization through a task replacement mechanism. However, its limitation lies in the fact that, while it can significantly improve the utilization of certain essential resources, it may also cause some tasks to be deferred due to the resource allocation strategy. For ex-

ample, tasks with short execution times but requiring multiple essential resources may be penalized and delayed, whereas tasks with long execution times but fewer essential resource requirements are less affected. As a result, although OPTI optimizes resource allocation, it still poses challenges in terms of fairness, especially when handling tasks with unbalanced resource demands.

IX. RESULTS DISCUSSION

A. *time_slot utilization summary performance*



1) FCFS maintains a relatively balanced but inflexible allocation strategy, with utilization rates ranging between ***23.4%*** and **26.6%** across different resources. While it prevents significant resource wastage, its strict queueing approach means that some slots remain idle when long-running tasks block new requests.
2) The priority-based nature of PRIO skews resource allocation. Lower-priority requests frequently experience delays, resulting in underutilization of available slots, with some resources dropping to as low as ***23.0%*** utilization. While PRIO ensures critical requests are handled promptly, it does so at the cost of inefficient slot distribution, leading to wasted capacity.
3) By dynamically adapting to task requirements, OPTI achieves the highest resource utilization levels, consistently reaching ***26.0%*** to ***26.6%*** across different services. This near-optimal usage suggests that fewer slots go unfilled, ensuring better distribution and preventing resource wastage. Compared to FCFS and PRIO, OPTI provides the most balanced and efficient resource management, making it the preferred choice for high-demand environments.

X. FUTURE WORK AND ALGORITHMIC ENHANCEMENTS

The optimization algorithm presented in this work demonstrates robust performance in balancing acceptance rates with resource utilization, yet several avenues exist for refinement and extension. While the current implementation achieves satisfactory results through its two-phase approach, further developments could enhance both theoretical rigor and practical applicability.

One key area for improvement lies in parameter optimization. Although the chosen parameters—initial weight calculation, penalty factors, and deferral thresholds—are grounded in established queueing theory and congestion control principles, their current values remain somewhat heuristic. A more systematic approach could involve large-scale simulation studies to identify optimal parameter sets for varying demand patterns. For instance, *machine learning techniques* might be employed to dynamically adjust penalty factors based on real-time system load, potentially improving both fairness and throughput. The relationship between penalty severity and essential scarcity could also be explored further, as current penalties apply uniformly across all resource types regardless of their individual utilization rates.

The algorithm's phased approach, while effective, presents opportunities for deeper optimization. The current two-phase process could be extended into a *multi-phases system* where deferred bookings undergo iterative reevaluation with progressively relaxed constraints. Such an approach might involve periodic reallocation windows where temporarily underutilized resources could be reassigned to deferred high-value bookings. This concept draws inspiration from preemptive scheduling systems but would require careful implementation to avoid excessive computational overhead. Additionally, introducing partial acceptance mechanisms—where bookings could be approved for shorter durations or fewer resources —might further improve overall system efficiency without significantly increasing complexity.

Scalability considerations warrant particular attention in future developments. While the algorithm performs well within the tested constraints of a one-week horizon with limited resources, real-world applications often operate at much larger scales. Extending the methodology to handle longer timeframes would likely require temporal partitioning strategies, where the scheduling problem is decomposed into overlapping windows to maintain computational tractability. Similarly, systems with substantially increased resource pools might benefit from hierarchical resource grouping or parallel processing architectures to maintain real-time responsiveness. These adaptations would need to preserve the algorithm's core advantages while accommodating expanded operational parameters.

Practical implementation challenges also merit investigation. Real-world systems must account for booking uncertainties, such as cancellations or timing flexibility, which could be addressed through stochastic optimization techniques. The integration of dynamic pricing models, where weights are adjusted based on demand fluctuations, could provide another dimension of optimization. Such enhancements would move the algorithm closer to commercial viability while maintaining its theoretical foundations.

The current implementation provides a strong foundation for resource-constrained scheduling, particularly in its ability to balance competing objectives through weighted prioritization. Future work should focus on bridging the remaining gaps between theoretical formulation and operational requirements, with particular attention to parameter sensitivity, phased optimization depth, and scalability. As resource-sharing systems grow in complexity and scale, these refinements could significantly enhance the algorithm's practical utility while preserving its computational efficiency and fairness guarantees.

XI. CONCLUSION

SmartPark represents a significant advancement over PolyU's legacy parking management system through its novel integration of operating system principles with practical scheduling heuristics. SPMS's multi-algorithm architecture demonstrates how OS-level process coordination enhances parking management through strategic scheduling. Our implementation leverages three complementary approaches, each optimized for specific operational scenarios.

The FCFS algorithm is simple and efficient, making it suitable for low-load environments where fairness is prioritized, such as batch processing or background task scheduling. However, since it does not consider task priorities, high-priority tasks may be delayed or even rejected if resources are occupied by earlier tasks. This makes FCFS unsuitable for real-time applications where responsiveness is crucial.

The PRIO algorithm addresses this issue by sorting tasks based on priority, ensuring that critical tasks are executed first. This improves resource utilization and guarantees timely execution of high-priority jobs. However, the sorting step increases complexity, and frequent task replacements may lead to delays for lower-priority tasks. In high-demand environments, this could cause resource starvation for non-critical jobs. Future improvements could focus on optimizing sorting efficiency and reducing unnecessary task replacements.

The OPTI algorithm combines PRIO's prioritization strategy with a task replacement mechanism to maximize resource utilization. While it significantly improves overall efficiency, it may penalize tasks requiring multiple essential resources, delaying their execution. Tasks with lower resource demands but longer durations may be favored, potentially leading to unfair scheduling. Future work could explore adaptive priority adjustments to ensure a more balanced allocation across varying task requirements.

SmartPark demonstrates how fundamental OS concepts —when carefully adapted—can solve modern resource management challenges in academic environments. Its implementation in the campus parking system suggests broader applicability to similar constrained allocation problems.

APPENDIX

A. *Source Code*

link: https://github.com/lalalalalalalaaaaaaaaa/SPMS_G24

B. *Pseudocodes of Three Scheduling Algorithms*

Algorithm 1: First-Come-First-Served (FCFS) Scheduling Algorithm

```
Input: Head node of original linked list: head
Output: Head node of processed linked list: new_head
if head is NULL then
    return NULL
end
Initialize new linked list: new_head ← NULL, new_tail ← NULL
Initialize traversal pointer: current ← head
while current is not NULL do
    Create new node new_node with copied booking data
    if check_resources(new_node.booking, new_head) is available then
        Set new_node.booking.status to ACCEPTED (1)
        Allocate resources for new_node.booking
    else
        Set new_node.booking.status to REJECTED (2)
    end
    if new_head is NULL then
        Set new_head ← new_node
        Set new_tail ← new_node
    else
        Link to tail: new_tail.next ← new_node
        Update tail: new_tail ← new_node
    end
    Move to next node: current ← current.next
end
return new_head
```

Algorithm 2: Priority Scheduling Algorithm

```
Input: Linked list of booking nodes input_list
Output: Scheduled list with priority order
Function schedulePriority(input_list)
    Reset all system resources;
    // Phase 1: Priority Sorting
    Create empty sorted_list;
    while input_list ≠ null do
        Extract current node n from input_list;
        Create new node with copied booking data;
        // Insert into sorted list (descending priority);
        if sorted_list = null or
         get_priority_value(n) > get_priority_value(sorted_list)
         then
            Insert at list head;
        else
            Traverse list to find insertion position;
            Insert while maintaining priority order;
    // Phase 2: Invoke FCFS Scheduling
    return scheduleFCFS(sorted_list);
```

Algorithm 3: Priority Scheduling Algorithm

```
Input: Linked list of booking nodes head
Output: Optimized scheduled linked list
Function scheduleOptimized(head)
    if head = null then
        return null;
    // Phase 1: Preparation
    Convert linked list to array nodes;
    count ← length of nodes;
    // Phase 2: Initial Sorting
    Sort nodes in descending order using compare_weights;
    // Phase 3: Dynamic Threshold
    threshold ← 0.75 × (average weight of nodes);
    // Phase 4: First Stage Allocation
    Initialize accepted_head ← null;
    Initialize deferred list;
    foreach node n in nodes do
        Calculate penalty based on time conflicts and common essentials
         with accepted nodes;
        n.weight ← n.weight × penalty;
        if n.weight ¡ threshold then
            Add n to deferred list;
            continue;
        if resources available then
            Add n to accepted_head;
            Update resources;
        else
            Mark n as rejected;
    // Phase 5: Process Deferred Requests
    Sort deferred list using compare_weights;
    foreach node d in deferred list do
        if resources available then
            Add d to accepted_head;
            Update resources;
        else
            Mark d as rejected;
    // Phase 6: Build Result List
    Construct result list by combining accepted and rejected nodes
     while preserving original order;
    return result list;
```

C. *Testing cases and summary reports*

Sample batch file:

```
addParking -member_A 2025-05-10 10:00 5.0 battery locker valetPark;
addReservation  -member_B 2025-05-10 08:00 13.0 locker;
bookESSentials  -member_B 2025-05-10 08:00 13.0 locker valetPark;
addParking -member_C 2025-05-10 10:00 6.0 battery locker valetPark;
```

a) *Request results by applying FCFS:*

When the FCFS algorithm is used, the only determining factor is the order in which the bookings are passed in. In the sample batch file, after member_B passes in the third booking, the locker resource is already exhausted. Hence, the SPMS system would reject member_C.

```
*** Parking Booking – ACCEPTED / fcfs ***

member_A has the following bookings:
Date         Start   End         Type         Device
=========================================================
2025-05-10   10:00   15:00       Parking      battery
                                              locker
                                              valetPark
                                              cable
                                              umbrella
                                              inflationService


member_B has the following bookings:
Date         Start   End         Type         Device
=========================================================
2025-05-10   08:00   21:00       Reservation  locker
                                              umbrella

2025-05-10   08:00   21:00       *            locker
                                              valetPark
                                              umbrella
                                              inflationService


- End -
=========================================================


*** Parking Booking – REJECTED / fcfs ***

member_C (there are 1 bookings rejected):
Date         Start   End         Type         Essentials
=========================================================
2025-05-10   10:00   16:00       Parking      battery
                                              locker
                                              valetPark
                                              cable
                                              umbrella
                                              inflationService


- End -
=========================================================
```

b) *Request results by applying PIRO:*

In the sample batch file, the priority of bookings made by member_B of bookEssentials is of the least hierarchy. Therefore, after the lockers are run out, the corresponding booking would be rejected.

```
*** Parking Booking – ACCEPTED / prio ***

member_A has the following bookings:
Date         Start   End         Type         Device
=========================================================
2025-05-10   10:00   15:00       Parking      battery
                                              locker
                                              valetPark
                                              cable
                                              umbrella
                                              inflationService


member_B has the following bookings:
Date         Start   End         Type         Device
=========================================================
2025-05-10   08:00   21:00       Reservation  locker
                                              umbrella


member_C has the following bookings:
Date         Start   End         Type         Device
=========================================================
2025-05-10   10:00   16:00       Parking      battery
                                              locker
                                              valetPark
                                              cable
                                              umbrella
                                              inflationService


- End -
=========================================================


*** Parking Booking – REJECTED / prio ***

member_B (there are 1 bookings rejected):
Date         Start   End         Type         Essentials
=========================================================
2025-05-10   08:00   21:00       *            locker
                                              valetPark
                                              umbrella
                                              inflationService


- End -
=========================================================
```

c) *Request results by applying OPTI:*

The scheduling process followed our two-phase optimization framework. The calculation process is explained in detail in Part IV.

```
*** Parking Booking – ACCEPTED / opti ***

member_B has the following bookings:
Date         Start   End         Type         Device
=========================================================
2025-05-10   08:00   21:00       Reservation  locker
                                              umbrella

2025-05-10   08:00   21:00       *            locker
                                              valetPark
                                              umbrella
                                              inflationService


member_C has the following bookings:
Date         Start   End         Type         Device
=========================================================
2025-05-10   10:00   16:00       Parking      battery
                                              locker
                                              valetPark
                                              cable
                                              umbrella
                                              inflationService


- End -
=========================================================


*** Parking Booking – REJECTED / opti ***

member_A (there are 1 bookings rejected):
Date         Start   End         Type         Essentials
=========================================================
2025-05-10   10:00   15:00       Parking      battery
                                              locker
                                              valetPark
                                              cable
                                              umbrella
                                              inflationService


- End -
=========================================================
```

d) *Comparisons:*

For FCFS, its strength lies in absolute fairness with linear efficiency. For PRIO, it ensures that urgent needs receive immediate responses through priority hierarchy and smart time-slot optimization. For OPTI,it dynamically adjusts request weights by penalizing shared resources in conflicting time slots and employs adaptive thresholding. After comparing the data obtained from several summary reports of different batch files, it is concluded that the utilization is higher by using the OPTI algorithm.

the summary report of the file above:

```
*** Parking Booking Manager – Summary Report ***
Performance:
 For FCFS:
    Total Number of Bookings Received: 4 (100.0%)
          Number of Bookings Assigned: 3 (75.0%)
          Number of Bookings Rejected: 1 (25.0%)
    Utilization of Time Slot:
    locker – 6.2%
    umbrella – 6.2%
    battery – 1.0%
    cable – 1.0%
    valetPark – 3.6%
    inflationservice – 3.6%
    Invalid request(s) made: 0
 For PRIO:
    Total Number of Bookings Received: 4 (100.0%)
          Number of Bookings Assigned: 3 (75.0%)
          Number of Bookings Rejected: 1 (25.0%)
    Utilization of Time Slot:
    locker – 4.8%
    umbrella – 4.8%
    battery – 2.2%
    cable – 2.2%
    valetPark – 2.2%
    inflationservice – 2.2%
    Invalid request(s) made: 0
 For OPTI:
    Total Number of Bookings Received: 4 (100.0%)
          Number of Bookings Assigned: 3 (75.0%)
          Number of Bookings Rejected: 1 (25.0%)
    Utilization of Time Slot:
    locker – 6.3%
    umbrella – 6.3%
    battery – 1.2%
    cable – 1.2%
    valetPark – 3.8%
    inflationservice – 3.8%
    Invalid request(s) made: 0
```

the summary report of a larger file:

```
*** Parking Booking Manager – Summary Report ***
Performance:
 For FCFS:
    Total Number of Bookings Received: 20 (100.0%)
          Number of Bookings Assigned: 12 (75.0%)
          Number of Bookings Rejected: 4 (25.0%)
    Utilization of Time Slot:
    locker – 12.9%
    umbrella – 12.9%
    battery – 12.9%
    cable – 12.9%
    valetPark – 12.9%
    inflationservice – 12.9%
    Invalid request(s) made: 4
 For PRIO:
    Total Number of Bookings Received: 20 (100.0%)
          Number of Bookings Assigned: 12 (75.0%)
          Number of Bookings Rejected: 4 (25.0%)
    Utilization of Time Slot:
    locker – 13.9%
    umbrella – 13.9%
    battery – 11.3%
    cable – 11.3%
    valetPark – 11.3%
    inflationservice – 11.3%
    Invalid request(s) made: 4
 For OPTI:
    Total Number of Bookings Received: 20 (100.0%)
          Number of Bookings Assigned: 12 (75.0%)
          Number of Bookings Rejected: 4 (25.0%)
    Utilization of Time Slot:
    locker – 17.7%
    umbrella – 17.7%
    battery – 9.9%
    cable – 9.9%
    valetPark – 17.7%
    inflationservice – 17.7%
    Invalid request(s) made: 4
```

REFERENCES

[1] L. Kleinrock, “Time-shared systems: A theoretical treatment,” vol. 11, no. 1. pp. 59–73, Mar. 1964.

[2] M. Harchol-Balter, “Performance Modeling and Design of Computer Systems: Queueing Theory in Action.,” no. 1. Cambridge University Press, Cambridge,UK, pp. 423–457, 2013.

[3] J. G. Dai and W. Lin, “Maximum Pressure Policies in Stochastic Processing Networks,” vol. 53, no. 2. pp. 197–218, Apr. 2005.

[4] A. Wierman and L. L. H. Andrew, “Fairness and Classifications,” vol. 37, no. 1. pp. 23–34, Jun. 2009.

[5] F. P. Kelly, “Charging and Rate Control for Elastic Traffic,” vol. 8, no. 1. pp. 33–37, Jan. 1997.

[6] S. Asmussen, “Applied Probability and Queues,2nd ed,” vol. 8, no. 1. New York, pp. 112–115, Jan. 2003.

[7] F. P. Kelly, “Reversibility and Stochastic Networks,” vol. 19, no. 1. pp. 45–62, Jan. 1982.

[8] A. Wierman, “Fairness and Scheduling in Single Server Queues,” vol. 16, no. 1. pp. 39–48, 2011.

[9] D. Bertsimas and I. C. Paschalidis, “Probabilistic Service Level Guarantees in Make-to-Stock Manufacturing Systems,” vol. 49, no. 1. pp. 119–133, Feb. 2001.

[10] M. Mitzenmacher, “The Power of Two Choices in Randomized Load Balancing,” vol. 12, no. 10. pp. 119–133, Feb. 2001.